

Tailwind & SASS



Intégration
Web

4ESGI Eductive - Master 1 Ingénierie du Web 2026

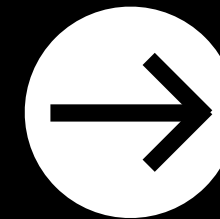
RNCP36469BC03 - B3 - Intégrer les architectures techniques et logicielles

Nicolas Hersant - Galactic Robots

Table Of Contents

Introduction à l'intégration web

Rôles et responsabilités de l'intégrateur web.
Présentation des outils.

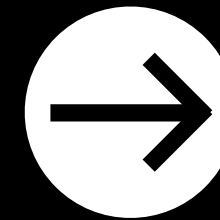


26/01

2H

Présentation de Tailwind CSS

Classes utilitaires, Création de composants.
Gestion des interactions utilisateur avec JavaScript.

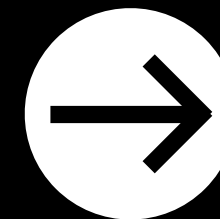


26/01
30/01

2 H 30
1 H 30

Introduction à SASS

Utilisation de variables, de mixins et de fonctions.
Organisation du code en modules.

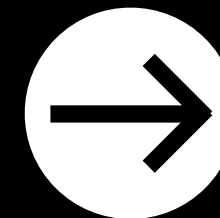


30/01
23/02

1 H 30
2 H 30

Optimisation et déploiement

Préparation des fichiers CSS pour le déploiement en production.
optimisation des performances et du temps de chargement.



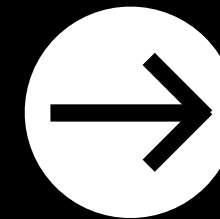
23/02

2 H

Table Of Contents

Examen final en groupe

2-3 personnes max / groupe
production des livrables demandés.

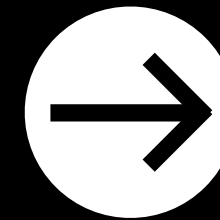


25/03

3 H

Soutenance finale

Présentation orale du travail réalisé



25/03

1 H 30

Sources

fr.wikipedia.org , tailwindcss.com , sass-lang.com , developer.mozilla.org , lighthouse , danielassani.com

Welcome To Presentation

On commence par un Quizz !

<https://www.menti.com/al6zvuz3nrxn>

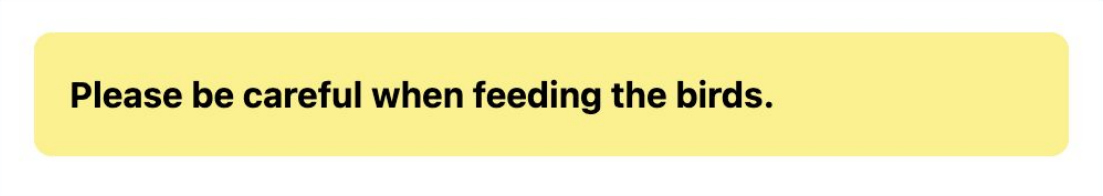


Tailwind

Tailwind CSS est un framework CSS open source.

Cette bibliothèque **ne procure pas une série de classes prédéfinies pour des éléments** tels que des boutons ou des tables. À la place, Tailwind crée une liste de **classes CSS « utilitaires »** pouvant être utilisés pour ajouter un style à **chaque élément** en les mélangeant et en les agencant.

Exemple: notice jaune

Résultat	
Code	<pre><div class="m-4 p-4 bg-yellow-200 font-bold rounded-lg"> <p>Please be careful when feeding the birds.</p> </div></pre>

<https://tailwindcss.com/>

SASS

Syntactically awesome stylesheets

Est un langage de script préprocesseur (SassScript) qui est compilé ou interprété en CSS, qui fournit : **variables, imbrication, mixins et héritage des sélecteurs.**

La « **syntaxe indentée** » pour séparer les blocs de code, les sauts de ligne séparent les directives. (.sass)
La nouvelle syntaxe, « **SCSS** » imbriquée, utilise les séparateurs de blocs CSS. (.scss)

SCSS	Sass
<pre>\$primary-color: #3bbfce; \$margin: 16px; .content-navigation { border-color: \$primary-color; color: darken(\$primary-color, 10%); } .border { padding: \$margin / 2; margin: \$margin / 2; border-color: \$primary-color; }</pre>	<pre>\$primary-color: #3bbfce \$margin: 16px .content-navigation border-color: \$primary-color color: darken(\$primary-color, 10%) .border padding: \$margin/2 margin: \$margin/2 border-color: \$primary-color</pre>

<https://sass-lang.com>

Intégration Web

Introduction 

Positionnement de l'intégration web dans une architecture logicielle moderne

L'intégration web ne se limite plus à la simple transformation de maquettes graphiques en pages HTML et CSS.

Dans les architectures modernes, l'intégrateur est un acteur à part entière de la chaîne de production logicielle. Il intervient à l'interface entre **le design, le développement applicatif et les contraintes techniques liées aux performances**, à **l'accessibilité** et à la **maintenabilité**.

Aujourd'hui, les interfaces web sont conçues comme des systèmes complexes et évolutifs. Elles doivent s'adapter à une grande variété de supports, intégrer des composants dynamiques, respecter des chartes graphiques parfois strictes et rester maintenables sur le long terme. **L'intégration devient ainsi une problématique d'architecture, au même titre que le backend ou l'infrastructure.**

Dans ce contexte, le CSS n'est plus un simple langage de mise en forme. Il est un **élément structurant du projet**, dont les choix techniques ont **un impact direct sur la qualité** globale du logiciel.



Transformer une intention graphique et fonctionnelle en une interface exploitable par navigateur.

Respecter des contraintes techniques.
Garantir la cohérence visuelle, la qualité de l'UX, la performance du rendu et la facilité d'évolution du code.

L'intégrateur moderne n'est pas isolé.
Il collabore avec les dev front et back, les designers UX/UI et parfois l'infra.
Ses décisions influencent directement la dette technique du projet.
(CSS difficile à faire évoluer)

responsable de bonnes pratiques :
responsive design, l'accessibilité, la gestion des états interactifs et l'optimisation des ressources statiques.

L'intégration web est désormais intégrée aux compétences RNCP liées aux architectures logicielles.



Rôles et responsabilités de l'intégrateur web

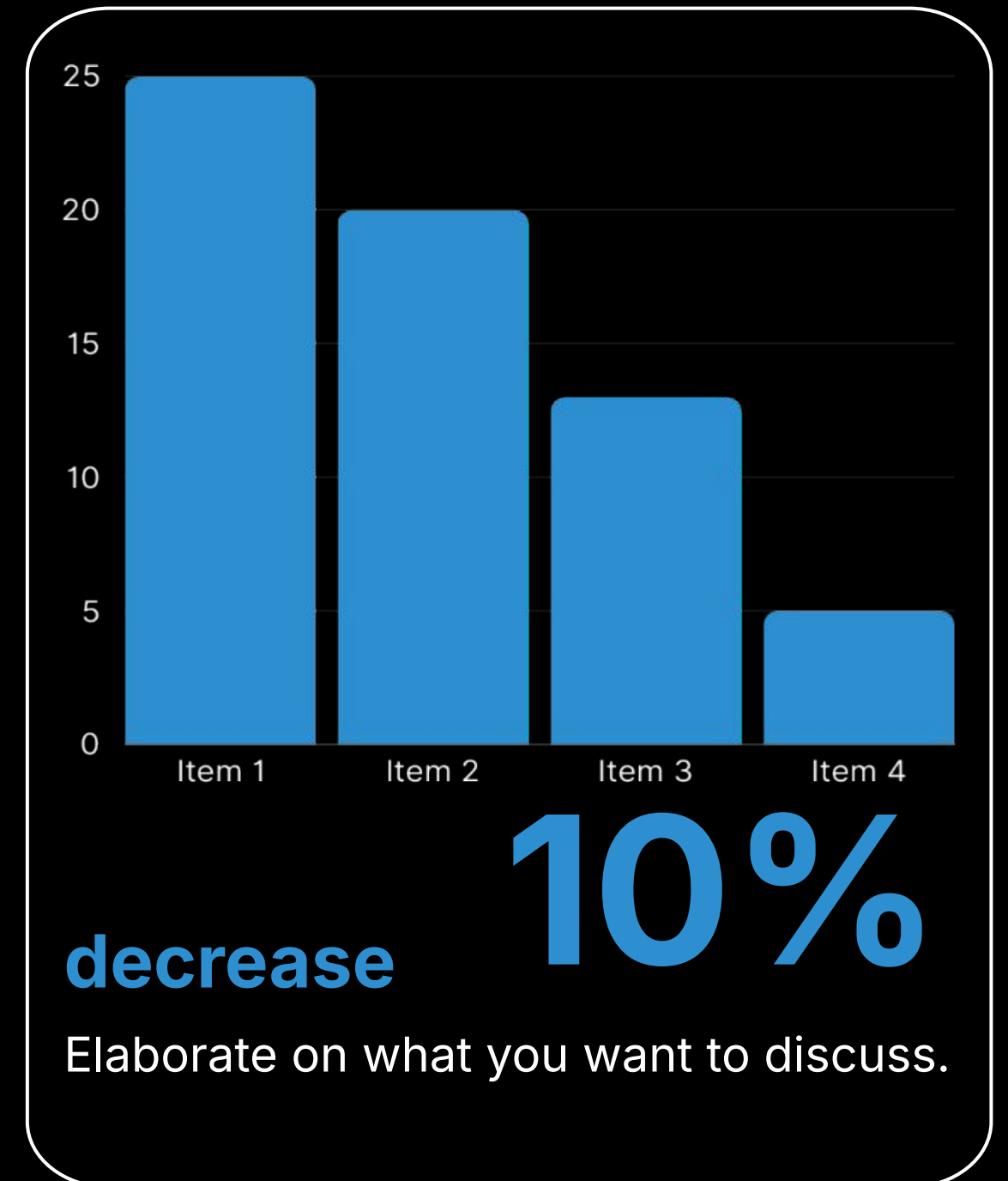
Problématiques récurrentes en CSS

Le CSS est l'un des premiers **facteurs de dette technique**.

L'absence de règles claires, la multiplication de fichiers ou au contraire, d'un fichier CSS monolithique, conduisent rapidement à **des effets de bord difficiles à maîtriser**.

Un CSS mal structuré est difficile à maintenir, car une modification locale peut avoir des conséquences imprévisibles sur l'ensemble de l'interface. La duplication de styles est fréquente, les conventions ne sont pas respectées et les correctifs successifs finissent par complexifier inutilement la base de code.

Dans les projets à long cycle de vie, impliquant plusieurs équipes, **le CSS devient un frein à l'évolution fonctionnelle**.



Panorama des approches d'intégration CSS

Le CSS natif reste la base du langage, mais il est souvent enrichi par des outils permettant d'améliorer la productivité et la maintenabilité

.css

CSS

utilisation native du langage sans surcouches. acceptable pour les petits projets, pages statiques, CV.

.scss

SASS / LESS

Ajoute des mécanismes d'abstraction, de modularité et de réutilisation, indispensables dans les projets de taille moyenne à grande.

.html

framework CSS traditionnel

Bootstrap ou Bulma fournissent des composants UI prédéfinis (boutons, cartes, barres de navigation) avec des styles figés. Le style consiste en quelques variables ou surcharges du CSS

.html

framework CSS Utility-First

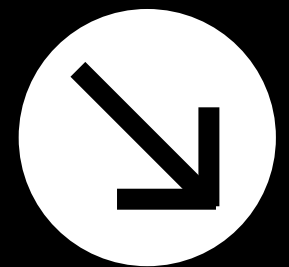
Logique d'assemblage d'interfaces à partir de classes atomiques, directement liées à des règles CSS précises. Accompagné d'utilitaires comme une configuration centralisée et une compilation optimisée.

Pourquoi associer Tailwind CSS et SASS

Tailwind CSS excelle dans la **mise en œuvre rapide de layouts**, de **composants** et de variations visuelles cohérentes grâce à son système de classes utilitaires et à sa configuration centralisée.

SASS, quant à lui, est particulièrement pertinent pour **structurer le code CSS**, **organiser les fichiers**, gérer des **abstractions complexes** et documenter l'architecture du style. Il permet de créer des **couches logiques** qui facilitent la compréhension et l'évolution du projet.

L'objectif de cette formation est précisément d'apprendre à utiliser ces **deux outils de manière complémentaire**, sans les opposer ni les utiliser de façon abusive. Cette capacité d'arbitrage est une compétence clé attendue à un niveau Master.





La Philosophie Utility-First

Objectifs

réduire la duplication de styles,
renforcer la cohérence visuelle,
limiter la dérive du CSS dans le temps.
éviter une surcharge de classes,
éviter une mauvaise lisibilité du code.

Concept

"Utility-First" est le pilier de Tailwind CSS.
Approche du front où la majorité du style
est appliquée sur les éléments HTML à
l'aide de classes utilitaires de faible niveau,
plutôt qu'à travers des feuilles de style CSS
externes écrites à la main.

```
<!-- Approche Traditionnelle -->
<button class="btn-primary">
  Cliquer ici
</button>
```

```
/* style.css */
.btn-primary {
  background-color: blue;
  color: white;
  padding: 10px 20px;
  border-radius: 5px;
  font-weight: bold;
}
```

```
- Approche Utility-First (Tailwind CSS) -->
<button class="bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded">
  Cliquer ici
</button>
```


Avantages de Utility-First



Rapidité du dev

- **Moins de changement de contexte.**
Le style est directement dans votre balisage HTML.
- **Nom de classes prédéfinis.**
Fini la nomenclature (BEM, SMACSS...).
- **Prototypage ultra-rapide.**
Création d'interfaces en un temps record.

Consistance du design

- **système de design prédéfini**
couleurs, espacements, tailles de police, breakpoints.
Garantie de cohérence visuelle.
- **Personnalisation centralisée**
dans `tailwind.config.js`.

Avantages de Utility-First

Maintenance facilitée

- Chaque classe n'affecte qu'un seul élément. Modifier un élément n'aura jamais d'effet de bord.
- Supprimer une classe sur un élément est sûr : cela ne cassera rien d'autre. Fini les fouilles angoissantes dans le code.

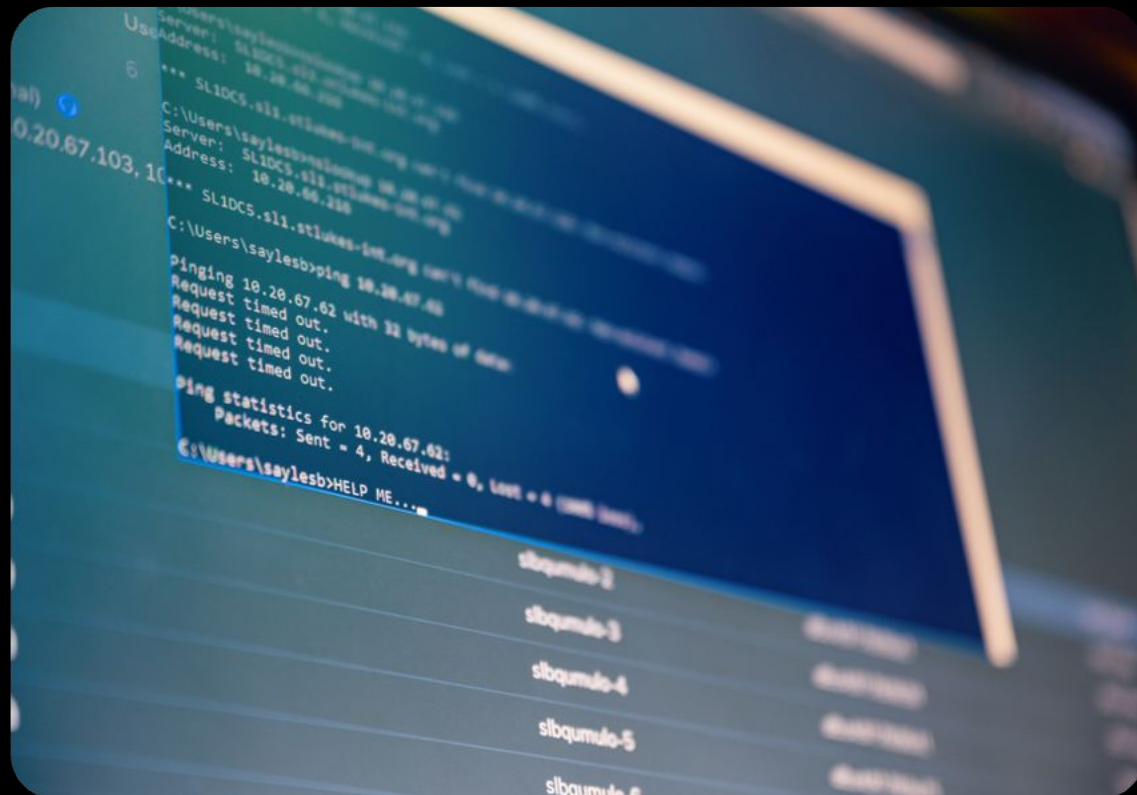
Optimisation du CSS

- Grâce à PostCSS et au moteur JIT (Just-In-Time) de Tailwind, le framework ne génère que le CSS réellement utilisé.
- Tailwind produit des fichiers CSS finaux extrêmement légers.



Inconvénient de Utility-First

HTML "pollué" / verbeux : C'est la critique fréquente. Votre balisage HTML peut sembler surchargé de classes.



Solution :

- **Habituez-vous !** C'est une courbe d'apprentissage visuelle. Avec le temps, ces classes deviennent intuitives.
- **Utilisez des composants :** Dans les frameworks modernes (React, Vue, Angular), vous encapsulez les patterns de classes répétitifs dans des composants réutilisables. Le HTML de votre composant sera verbeux, mais le HTML *qui l'utilise* sera propre.
- `@apply` : Pour les cas exceptionnels où vous avez des styles utilitaires très répétés et que vous ne pouvez pas (ou ne voulez pas) utiliser un composant JavaScript, Tailwind propose la directive `@apply` pour extraire des classes utilitaires vers une nouvelle classe CSS.



Travaux pratiques

Installation d'un nouveau projet avec vite et de TailwindCSS

tailwindcss.com/docs/installation

github.com/nicoHersant/m1-tailwindSASS

Objectifs :

Vérifier une installation Tailwind fonctionnelle avec Vite

Comprendre le rôle des classes utilitaires de layout

Construire un layout simple et responsive

Appliquer flex et grid

tailwindcss.com/docs/styling-with-utility-classes

Exemples de classes utilitaires :

- ``p-4`` : Applique un ``padding`` de 16px tout autour (par défaut, Tailwind utilise une échelle de 4px pour le padding/margin).
- ``text-lg`` : Définit la taille du texte à "large".
- ``flex`` : Rend l'élément un conteneur flexbox.
- ``bg-blue-500`` : Définit la couleur de fond en bleu (nuance 500 de la palette de couleurs par défaut de Tailwind).
- ``shadow-md`` : Applique une ombre moyenne à l'élément.

Terminal

```
npm create vite@latest my-project  
cd my-project
```

Terminal

```
npm install tailwindcss @tailwindcss/vite
```

```
npm install -D tailwindcss postcss autoprefixer
```

```
npm install tailwindcss @tailwindcss/cli
```


Explication des classes données en exemple (repo git):

max-w-sm : Limite la largeur maximale de la carte à une taille "small" sur mobile.

mx-auto : Centre la carte horizontalement (margin-left: auto; margin-right: auto;).

bg-white, rounded-xl, shadow-md, overflow-hidden : Stylisation de base (fond blanc, bords arrondis très prononcés, ombre moyenne, cache le contenu qui déborde).

md:max-w-2xl : Sur les écrans de taille moyenne (breakpoint md:), la largeur maximale passe à une taille plus grande (environ 48rem ou 768px). Ceci est un exemple de **design responsif directement intégré**.

my-8 : Applique une marge verticale (margin-top et margin-bottom) de 32px.

md:flex : Sur les écrans de taille moyenne et plus grands, les enfants de cet élément (<div> de l'image et <div> du contenu) seront disposés en flexbox, côte à côte. Sur mobile, ils restent empilés verticalement par défaut.

md:shrink-0 : Empêche l'image de rétrécir sur les écrans moyens.

h-48 w-full : Sur mobile, l'image a une hauteur de 192px et prend toute la largeur disponible.

object-cover : Assure que l'image couvre entièrement son conteneur sans être déformée.

md:h-full md:w-48 : Sur les écrans moyens, l'image prend toute la hauteur de son parent flex et une largeur fixe de 192px.

p-8 : Applique un padding de 32px tout autour du contenu.

block : L'ancre se comporte comme un bloc.

mt-1 : Marge supérieure de 4px.

text-lg, leading-tight, font-medium, text-black : Styles de texte (taille, hauteur de ligne compacte, gras moyen, couleur noire).

hover:underline : Souligne le texte au survol.

mt-4 : Marge supérieure.

flex items-center justify-between : Utilise flexbox pour aligner verticalement les éléments au centre et les espacer au maximum horizontalement.

liste des fonctions dépréciées dans la V4 Tailwind
<https://tailwindcss.com/docs/upgrade-guide#removed-deprecated-utilities>



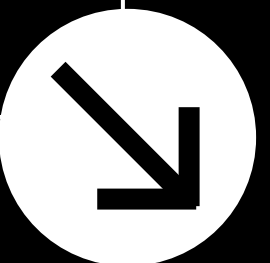
Explication du `tailwind.config.js`

content : C'est la propriété cruciale de Tailwind CSS. Elle indique à Tailwind quels fichiers il doit scanner pour détecter les classes que vous utilisez. C'est grâce à cela que Tailwind peut générer uniquement le CSS nécessaire (ce qu'on appelle "tree-shaking" ou "purging") et ne pas inclure des classes non utilisées.

theme : Cette section vous permet de définir votre propre système de design. Vous pouvez y étendre (extend) ou remplacer les valeurs par défaut de Tailwind pour les couleurs, les tailles d'espacement, les polices, les breakpoints, etc.

extend : Permet d'ajouter de nouvelles valeurs ou de modifier des valeurs existantes sans supprimer les valeurs par défaut de Tailwind.

plugins : Ici, vous pouvez ajouter des plugins Tailwind officiels ou communautaires, ou même écrire vos propres plugins pour ajouter de nouvelles fonctionnalités CSS ou des utilitaires personnalisés.





Flexbox

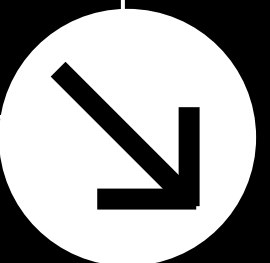
Layout Unidimensionnel

Explication de Flexbox

Flexbox est un **module de layout** CSS conçu pour organiser des éléments le long d'une seule dimension (une ligne ou une colonne). C'est l'outil parfait pour distribuer l'espace entre les éléments d'un conteneur, les aligner et contrôler leur ordre.

Concepts Fondamentaux de Flexbox

- **Conteneur Flex** (Flex Container) : L'**élément parent** sur lequel vous appliquez **display: flex** (ou **display: inline-flex**). C'est lui qui devient le contexte de Flexbox pour ses enfants directs.
- **Éléments Flex** (Flex Items) : Les **enfants directs** du conteneur Flex. Ce sont ces éléments qui seront arrangés par Flexbox.
- **Axe Principal** (Main Axis) : L'axe le long duquel les **éléments flex sont disposés**. Par défaut, il est horizontal (gauche à droite).
- **Axe Transversal** (Cross Axis) : L'axe **perpendiculaire à l'axe principal**. Par défaut, il est vertical (haut en bas).



Propriétés Clés des Conteneur Flex

display : Définit un conteneur flex.

CSS : *display: flex;*

Tailwind : *flex*

flex-direction : Définit la direction de l'axe principal, en ligne ou en colonne.

CSS : *flex-direction: row; (par défaut), column;, row-reverse;, column-reverse;*

Tailwind : *flex-row, flex-col, flex-row-reverse, flex-col-reverse*

justify-content : Aligne les éléments le long de l'axe principal.

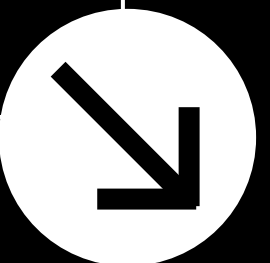
CSS : *justify-content: flex-start;, flex-end;, center;, space-between;, space-around;, space-evenly;*

Tailwind : *justify-start, justify-end, justify-center, justify-between, justify-around, justify-evenly*

align-items : Aligne les éléments le long de l'axe transversal.

CSS : *align-items: flex-start;, flex-end;, center;, baseline;, stretch;*

Tailwind : *items-start, items-end, items-center, items-baseline, items-stretch*



Propriétés Clés des Conteneur Flex

flex-wrap : Spécifie si les éléments flex doivent passer à la ligne s'ils ne tiennent pas sur une seule ligne.

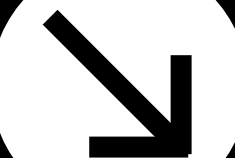
CSS : *flex-wrap: nowrap; (par défaut), wrap;, wrap-reverse;*

Tailwind : *flex-nowrap, flex-wrap, flex-wrap-reverse*

gap : Définit l'espace entre les éléments flex.

CSS : *gap: 16px; ou gap: 1rem 2rem;*

Tailwind : *gap-4 (pour 16px), gap-x-8, gap-y-4*



Propriétés Clés des Éléments Flex

flex-grow : Définit la capacité d'un élément à s'agrandir pour occuper l'espace disponible.

CSS : *flex-grow: 1; (pour s'agrandir), 0; (pour ne pas s'agrandir)*

Tailwind : **flex-grow, flex-grow-0**

flex-shrink : Définit la capacité d'un élément à rétrécir si l'espace est insuffisant.

CSS : *flex-shrink: 1; (pour rétrécir), 0; (pour ne pas rétrécir)*

Tailwind : **flex-shrink, flex-shrink-0**

flex-basis : Définit la taille initiale d'un élément avant que l'espace restant ne soit distribué.

CSS : *flex-basis: 200px; ou flex-basis: auto;*

Tailwind : **basis-1/2, basis-full, basis-auto**

order : Contrôle l'ordre visuel des éléments flex, indépendamment de leur ordre dans le DOM.

CSS : *order: 1;*

Tailwind : **order-first, order-last, order-none, order-1, order-2, etc.**



Propriétés Clés des Éléments Flex

align-self : Permet d'aligner un élément le long de l'axe transversal, écrasant align-items du conteneur parent.

CSS : *align-self: center;*

Tailwind : *self-center, self-start, self-end, etc.*



Menu principale verticale fixé à la partie gauche de l'écran.

Une série de liens (icone + texte) servant de navigation avec des effets de hover et selected.

Header fixe avec un bouton permettant d'afficher ou rétrécir le menu principale.

en mode rétrécit le menu n'affiche que les icônes.

Les menu et header sont fixent à tout moment de l'application.

Seul la partie de contenu <main> peut scroller en fonction de la taille des contenus.

suite du TP : Initialisation d'un Dashboard



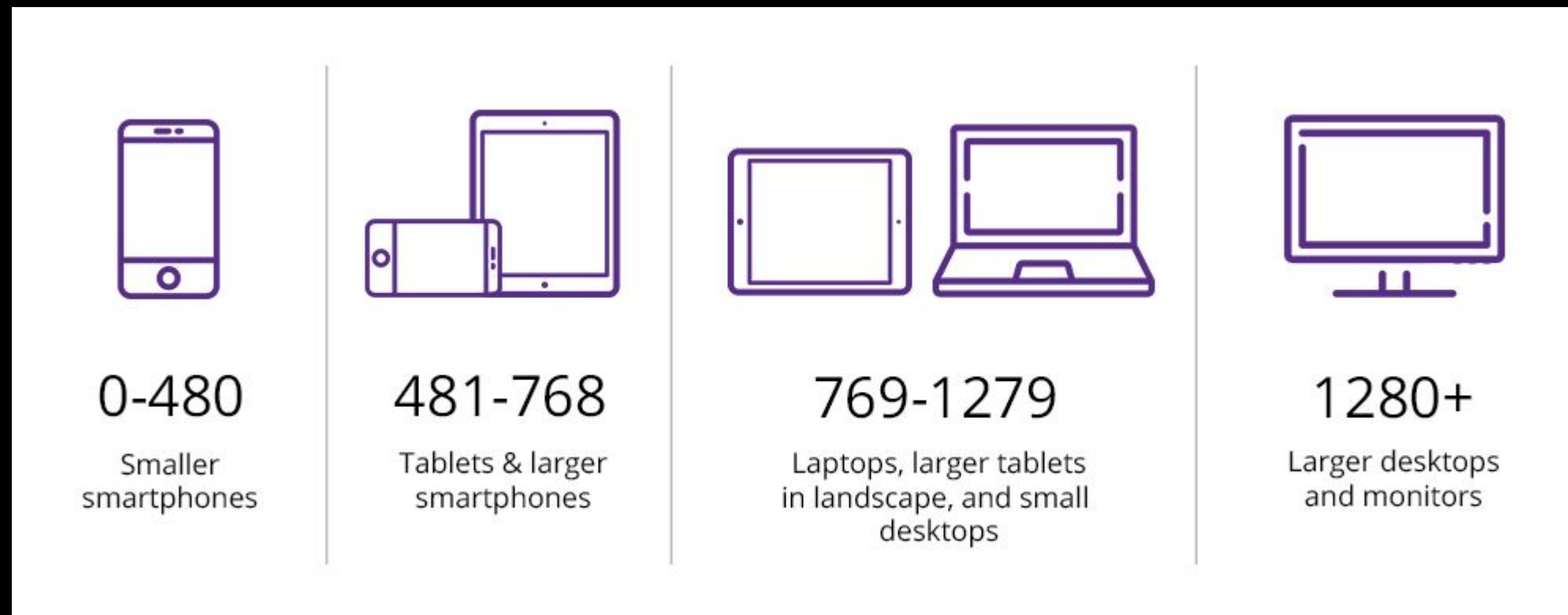


Grid

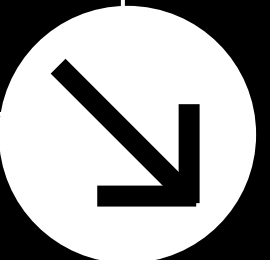
Layout Bidimensionnel

Breakpoints classics

Comparaison avec Bootstrap :



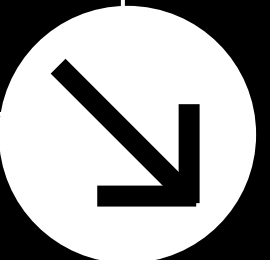
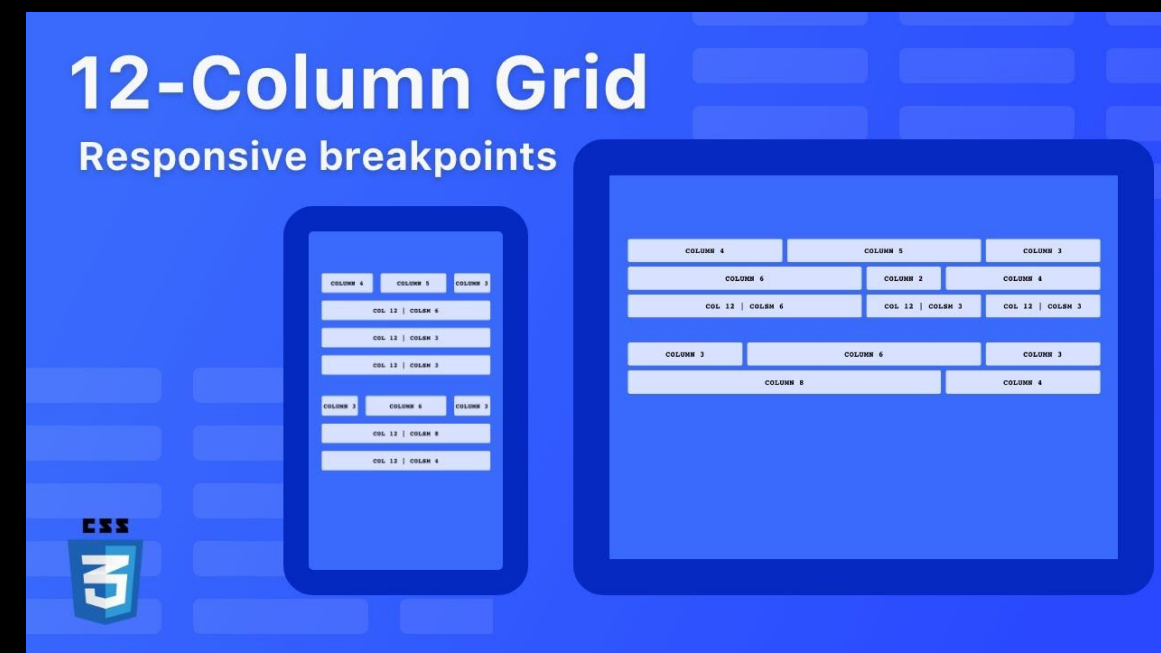
Breakpoint	Class infix	Dimensions
X-Small	-	<576px
Small	sm	≥576px
Medium	md	≥768px
Large	lg	≥992px
X-Large	xl	≥1200px
XX-Large	xxl	≥1400px



Explication de Grid

CSS Grid est un système de layout beaucoup plus puissant que Flexbox, car il permet d'organiser des éléments dans deux dimensions simultanément : en lignes et en colonnes. C'est l'outil de prédilection pour la structure globale d'une page ou des composants complexes qui nécessitent un alignement précis sur les deux axes.

```
<section class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4">
  <div class="bg-white p-4 shadow">Carte 1</div>
  <div class="bg-white p-4 shadow">Carte 2</div>
  <div class="bg-white p-4 shadow">Carte 3</div>
  <div class="bg-white p-4 shadow">Carte 4</div>
</section>
```



Explication de Grid

Conteneur Grid (Grid Container) : L'élément parent sur lequel vous appliquez `display: grid`.

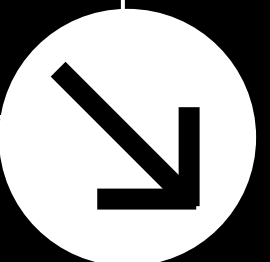
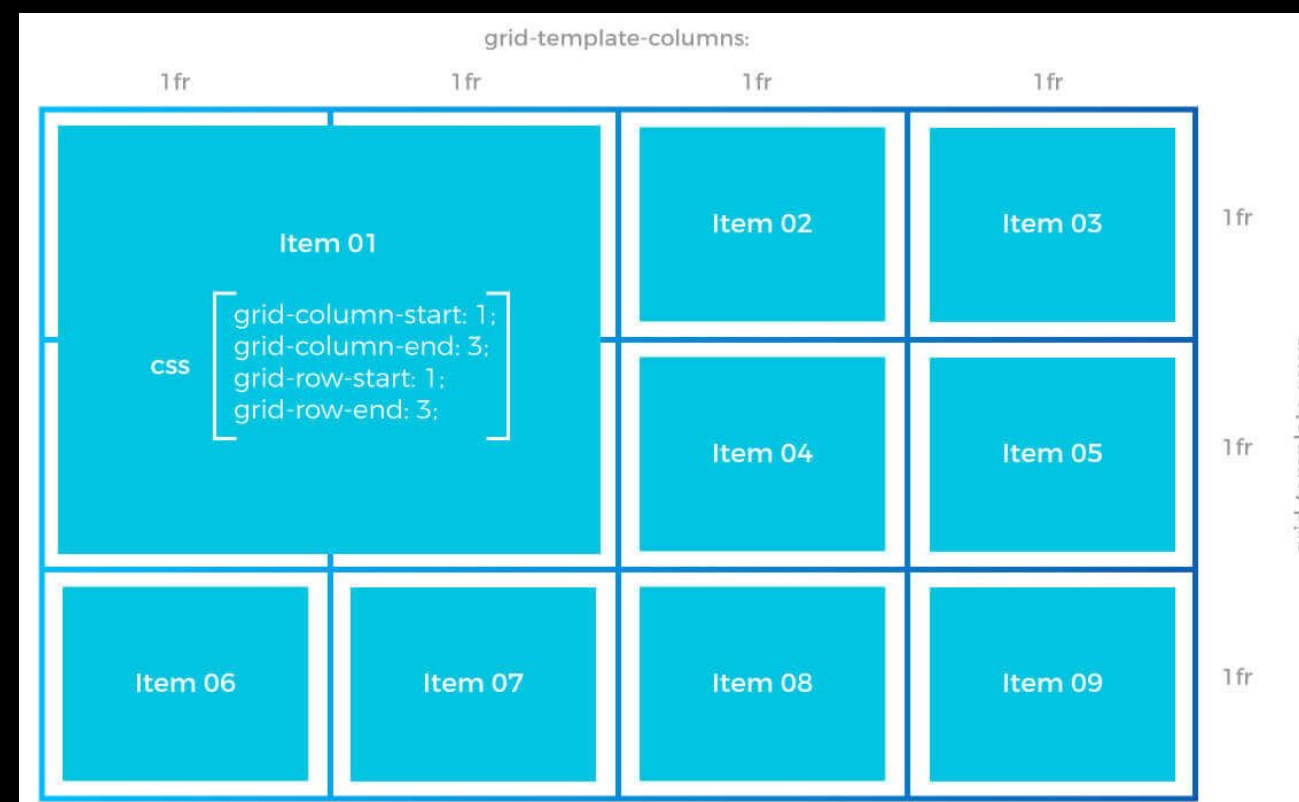
Éléments Grid (Grid Items) : Les enfants directs du conteneur Grid.

Lignes Grid (Grid Lines) : Les lignes horizontales et verticales qui composent la grille.

Pistes Grid (Grid Tracks) : Les espaces entre deux lignes de grille consécutives (col & row).

Cellules Grid (Grid Cells) : L'intersection d'une ligne et d'une colonne de grille, formant une case.

Zones Grid (Grid Areas) : Des blocs nommés qui couvrent plusieurs cellules de la grille, permettant de positionner des éléments par leur nom.



Propriétés Clés des Conteneur CSS Grid

display : Définit un conteneur grid.

CSS : *display: grid;*

Tailwind : `grid`

grid-template-columns : Définit le nombre et la taille des colonnes de la grille.

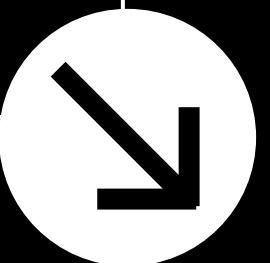
CSS : *grid-template-columns: 1fr 1fr 1fr; ou repeat(3, 1fr); ou 200px 1fr auto;*

Tailwind : `grid-cols-3`, `grid-cols-none`, `grid-cols-12` (pour 12 colonnes implicites),
`grid-cols-[1fr_200px_auto]` (pour des valeurs personnalisées)

grid-template-rows : Définit le nombre et la taille des lignes de la grille.

CSS : *grid-template-rows: 100px 1fr auto;*

Tailwind : `grid-rows-3`, `grid-rows-none`, `grid-rows-[100px_1fr_auto]`



Propriétés Clés des Conteneur CSS Grid

gap (ou grid-gap) : Définit l'espace entre les lignes et les colonnes.

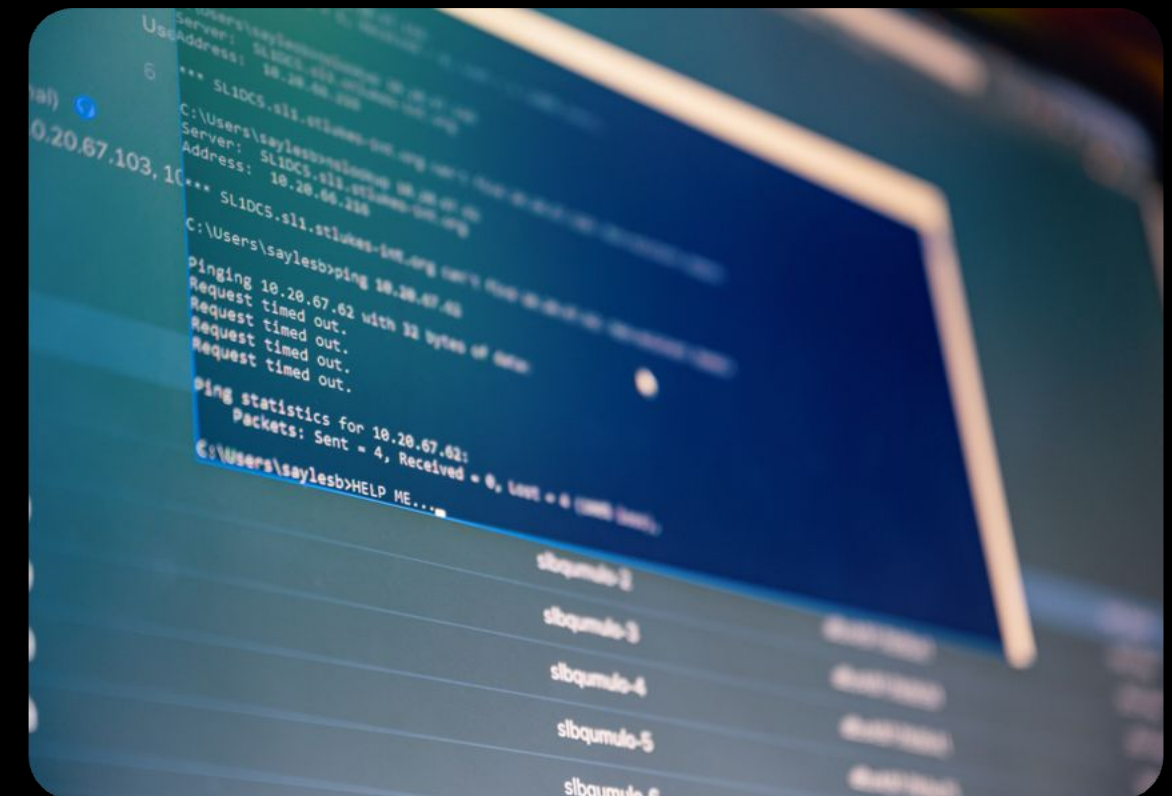
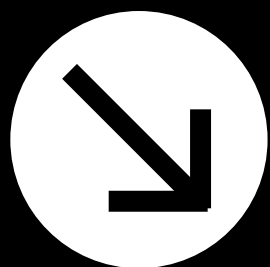
CSS : `gap: 1rem; ou row-gap: 1rem; column-gap: 2rem;`

Tailwind : `gap-4, gap-x-8, gap-y-4`

grid-auto-flow : Contrôle comment les éléments auto-placés sont insérés dans la grille.

CSS : `grid-auto-flow: row;, column;, dense;`

Tailwind : `grid-flow-row, grid-flow-col, grid-flow-dense`



Propriétés Clés des Éléments CSS Grid

grid-column : Spécifie la colonne de début et de fin d'un élément.

CSS : `grid-column: 1 / 3;` (de la ligne 1 à la ligne 3), `span 2;` (sur 2 colonnes)

Tailwind : `col-span-2`, `col-start-1`, `col-end-3`, `col-auto`

grid-row : Spécifie la ligne de début et de fin d'un élément.

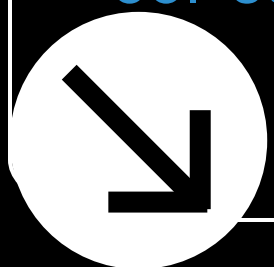
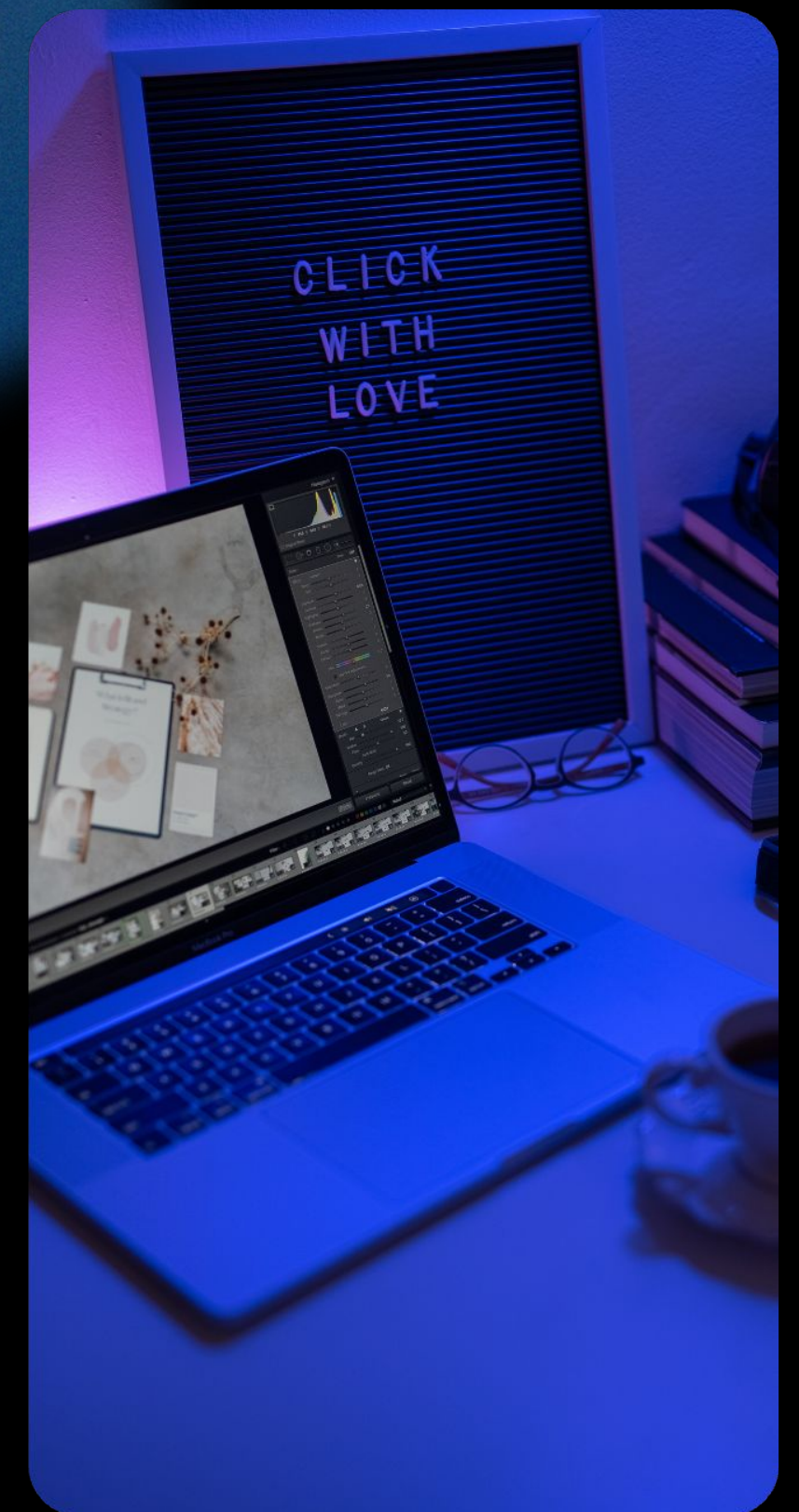
CSS : `grid-row: 2 / 4;`

Tailwind : `row-span-2`, `row-start-2`, `row-end-4`, `row-auto`

grid-area : Nomme un élément ou le place dans une zone nommée.

CSS : `grid-area: header;`

Tailwind : Nécessite une configuration personnalisée dans `tailwind.config.js` pour utiliser les noms de zones, ou utiliser les classes `col-start/end` et `row-start/end`.



Combiner Flexbox et Grid

La vraie puissance réside dans l'art de combiner Flexbox et Grid.
Ils ne sont pas mutuellement exclusifs ; ils sont complémentaires.
Répartir les rôles entre flex et grid permet d'élaborer des layouts avancés.

Grid pour la macro-disposition

La structure globale de la page
(en-tête, barre latérale, contenu principal, pied de page).

Grid excelle à **positionner des blocs majeurs** de contenu dans une **grille bidimensionnelle**.

Flexbox pour la micro-disposition

L'arrangement des éléments à l'intérieur d'une cellule de la grille.

Flexbox est idéal pour aligner les éléments de contenus et distribuer l'espace entre eux.



En-tête de page :

Le header est une cellule de grille qui s'étend sur toute la largeur. À l'intérieur du header, vous pouvez utiliser Flexbox pour aligner le logo à gauche et les éléments de navigation à droite.

Cartes de produits :

Un main content est une cellule de grille. À l'intérieur, vous avez un conteneur Flexbox qui gère l'affichage des cartes de produits, avec un nombre variable de cartes par ligne en fonction de l'espace.

Contenu d'une section :

Une section peut être une cellule de grille. À l'intérieur, si vous avez des éléments qui doivent être alignés horizontalement (comme des icônes et du texte), Flexbox est parfait.

Scénarios typiques



Suite du TP

- Reprendre le layout avec 3 sections principales en grid : menu, header et main
- Reprendre le contenu de chaque section avec flexbox
- Ajouter du contenu à main : des cards et un footer pour confirmer l'effet de scroll.
- Ajouter une gestion uniforme du responsive avec les breakpoints tailwind

- ``sm`` : 640px
- ``md`` : 768px
- ``lg`` : 1024px
- ``xl`` : 1280px
- ``2xl`` : 1536px



Design system & composants Tailwind

Design tokens

Les design tokens sont des **valeurs de référence centralisées**.

Ils décrivent les **choix fondamentaux de l'interface** : couleurs, tailles de texte, espacements, rayons de bordure, ombres...

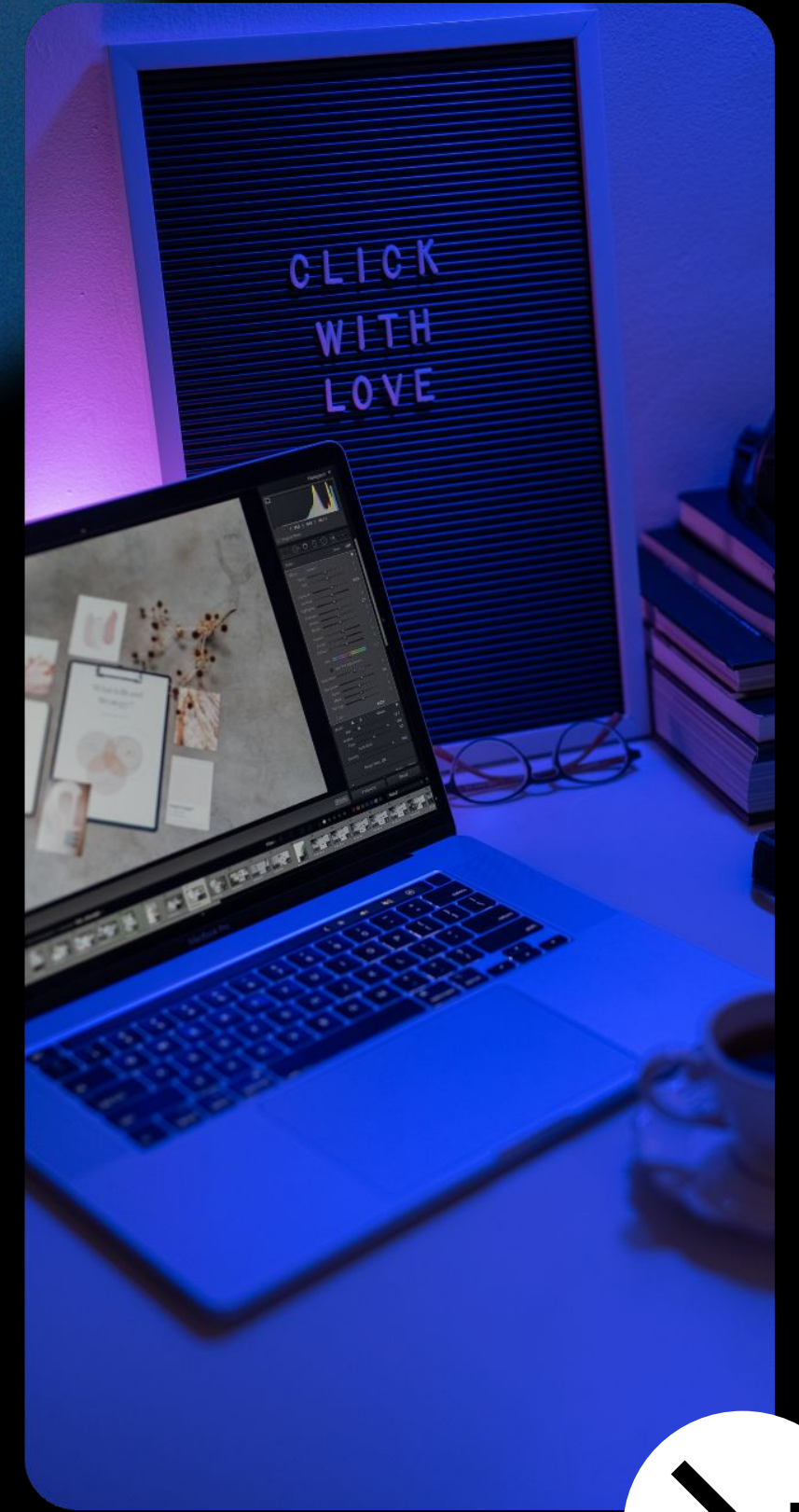
Ils ne décrivent pas des composants, mais des **contraintes de design** partagées par toute l'application.

Dans Tailwind, ces tokens sont définis dans **tailwind.config.js**.

Cela signifie que **chaque classe utilitaire** (bg-primary-500, text-sm, p-4) est une **projection directe d'un token**, et non une décision locale arbitraire.

- cohérence visuelle globale
- suppression des "valeurs magiques"
- facilité de refonte graphique
- alignement design / intégration / produit

Un token doit être sémantique, pas descriptif. On préfère primary, secondary, danger à blue, red, etc, car le sens métier survit aux changements graphiques.



Single Source of Truth (SSOT)

Le principe consiste à **définir un seul endroit où la vérité graphique existe.**

Dans une intégration Tailwind, cette source unique est principalement :

- tailwind.config.js pour les valeurs
- le HTML pour la composition visuelle

Il ne doit pas exister :

- plusieurs définitions concurrentes de la même couleur
- plusieurs tailles "presque identiques"
- des décisions visuelles prises localement sans justification

Ce principe permet d'éviter la dérive visuelle, de limiter les débats arbitraires et de sécuriser le travail en équipe.

Sans SSOT, un design system n'est qu'un catalogue décoratif.



Composants visuels ≠ composants framework

Un composant visuel est une **convention d'assemblage de classes** Tailwind, pas forcément un composant React, Vue ou autre.

Exemple :

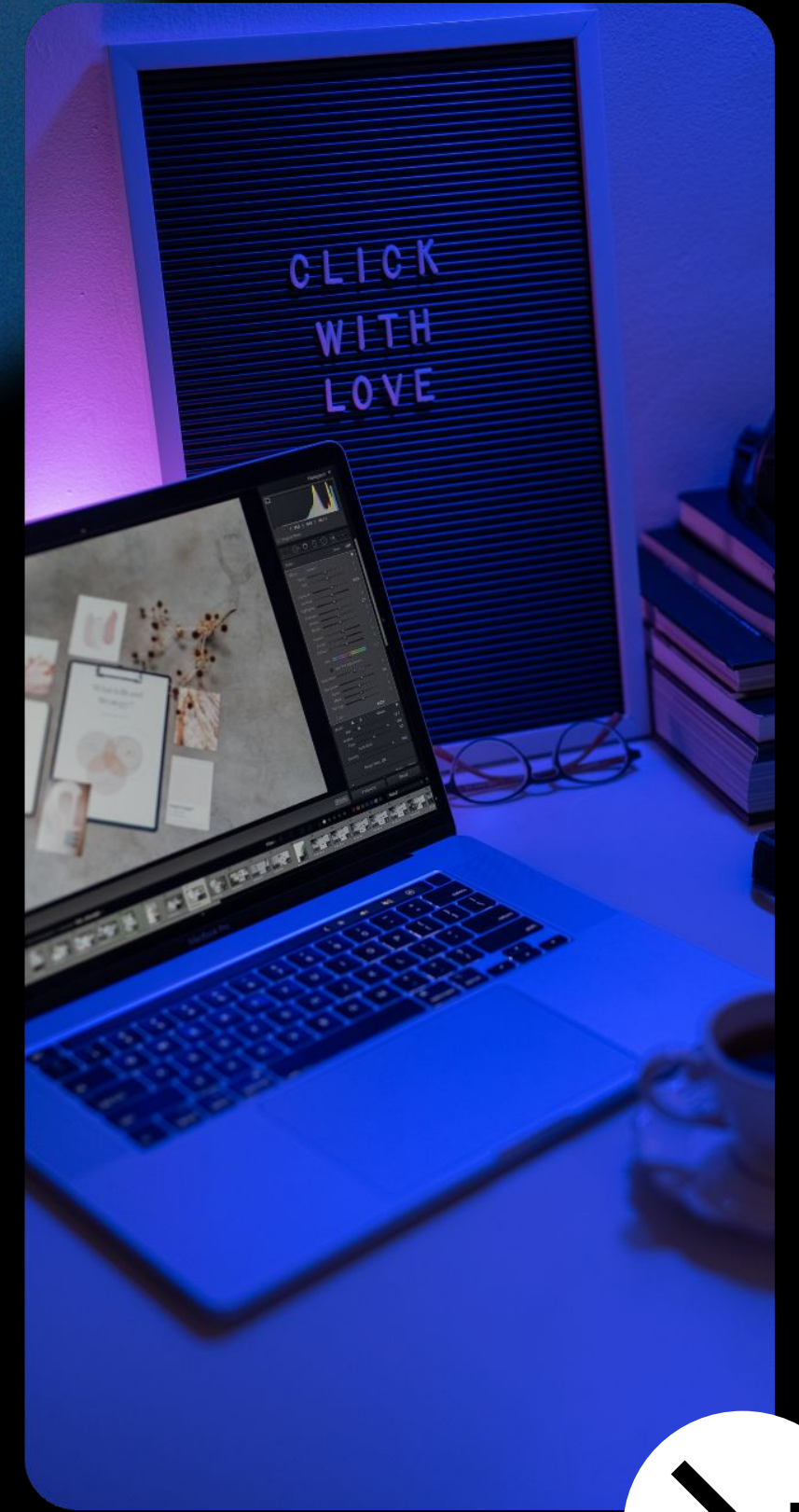
- un bouton primaire est défini par un pattern de classes
- il peut être copié, factorisé, documenté
- mais il n'a pas besoin d'être immédiatement abstrait dans un composant JS

L'erreur fréquente est de vouloir **créer trop tôt des composants** techniques, ou figer des choix encore instables qui vont **complexifier** inutilement **l'architecture**.

Tant que la variation est faible et lisible, rester en HTML + classes Tailwind.

L'abstraction ne devient pertinente que lorsque :

- la **duplication** devient coûteuse
- les **variations** deviennent nombreuses
- le **comportement** dépasse le simple style



Variantes (interaction, thème, contexte)

Une variante est une **modification conditionnelle du style**, déclenchée par :

- une interaction utilisateur (hover, focus)
- un état (disabled)
- un contexte (dark, group, peer)
- un breakpoint (md, lg)

Tailwind rend ces variantes explicites dans la classe elle-même :

- le comportement est lisible dans le HTML
- aucune règle CSS cachée n'existe ailleurs

"Je lis le HTML, je comprends comment le composant réagit."

Les variantes ne doivent pas être empilées sans réflexion.

Une multiplication excessive est souvent le signe d'un composant mal défini ou d'un besoin d'abstraction supérieur.



Lisibilité > abstraction prématurée

Tailwind accepte volontairement une certaine verbosité dans le HTML, car cette **verbosité est explicite, localisée, sans effet de bord.**

À l'inverse, une abstraction trop précoce :

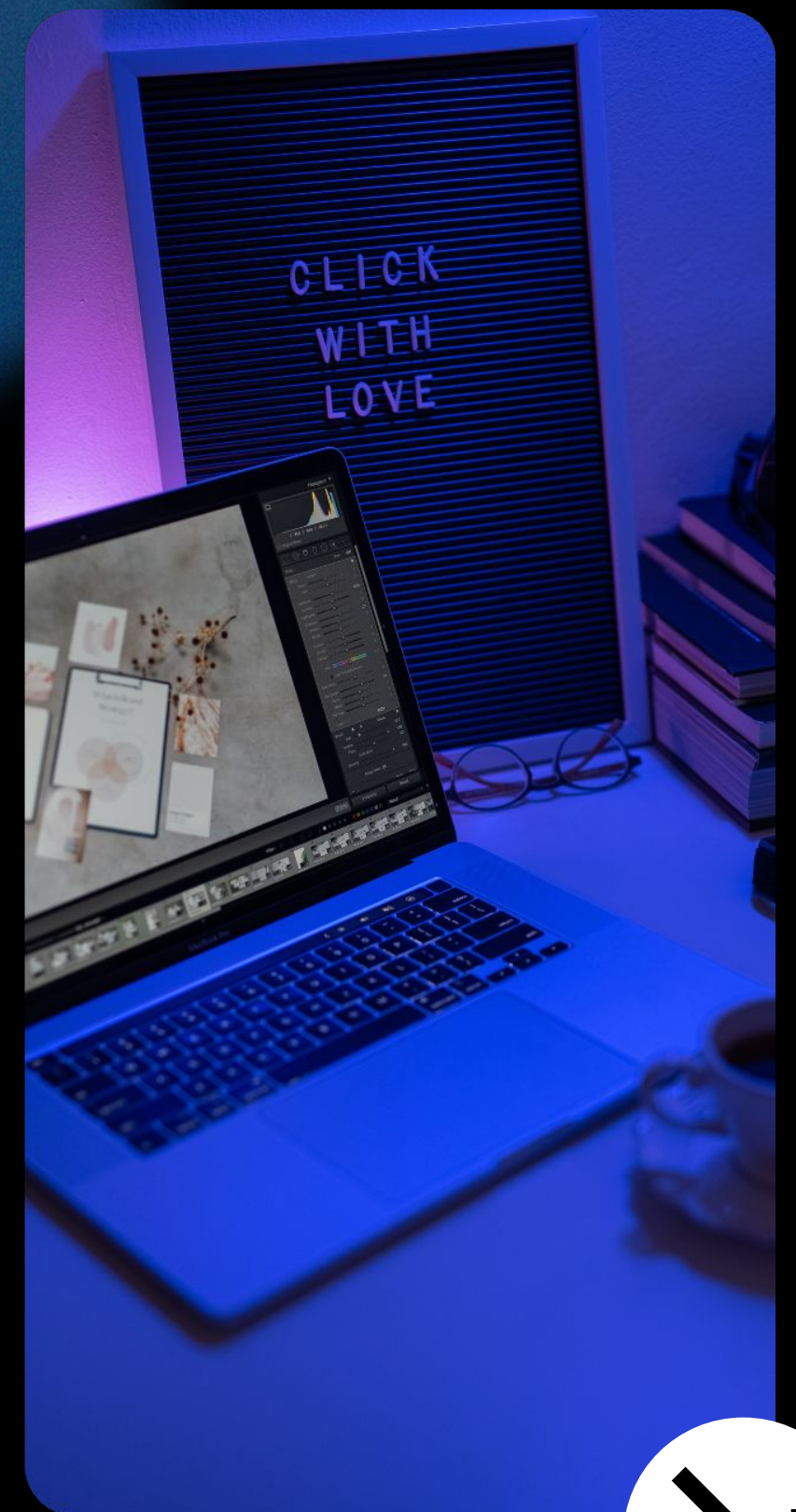
- masque la logique visuelle
- rend le débogage plus difficile
- augmente la dépendance entre fichiers

Règle opérationnelle :

on privilégie la lisibilité tant que le composant est simple

on abstrait quand le coût cognitif devient supérieur au gain

C'est exactement cette capacité d'arbitrage qui est attendue à un niveau Master.





Quand introduire @apply avec Tailwind CSS

C'est quoi @apply ?

directive CSS qui permet d'assembler plusieurs classes utilitaires Tailwind dans une classe CSS personnalisée.

Qu'est ce que ca fait ?

Tailwind remplace simplement les utilitaires par leurs règles CSS équivalentes au build.

Erreur fréquente :

L'erreur la plus courante consiste à introduire @apply dès les premiers composants, pour "faire plus propre".

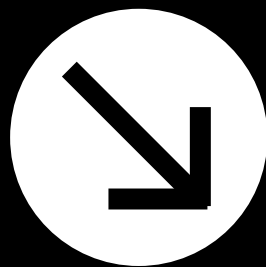
- on perd la lisibilité immédiate
- on masque la structure visuelle
- on recrée une abstraction CSS classique
- on annule l'intérêt principal de Tailwind

```
.btn-primary {  
  @apply px-4 py-2 rounded bg-primary-500 text-white hover:bg-primary-700  
}
```


Utiliser @apply

Toujours placer @apply dans des fichiers dédiés (components.css, utilities.css)

- Ne jamais utiliser @apply pour tout le projet
- Éviter les classes trop génériques (.card, .box)
- Garder un lien clair avec les design tokens

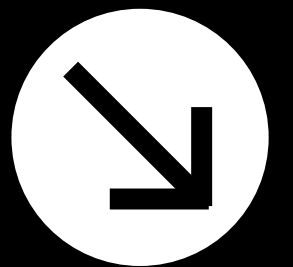


Ne pas utiliser @apply

le composant n'existe qu'une ou deux fois

- la variation est fréquente
- le style dépend fortement du contexte
- le HTML reste parfaitement lisible

Dans ces cas, Tailwind pur est plus expressif.





SASS & architecture CSS

Rôle de SASS dans un projet Tailwind

SASS n'est **pas** là pour remplacer Tailwind.

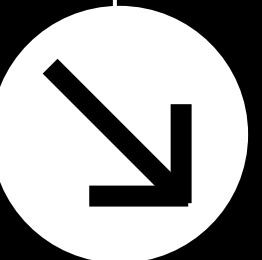
Il intervient là où Tailwind s'arrête :

- organisation des fichiers
- factorisation de patterns
- logique conditionnelle
- documentation implicite de l'architecture

Dans un projet professionnel, SASS est un outil d'architecture,
pas un outil de décoration.



```
npm install -D sass
```



SCSS

```
1 section {
2   height: 100px;
3   width: 100px;
4 }
5
6 .class-one {
7   height: 50px;
8   width: 50px;
9 }
10
11 .button {
12   color: #074e68;
13 }
```

CSS

```
1 section {
2   height: 100px;
3   width: 100px;
4 }
5
6 section .class-one {
7   height: 50px;
8   width: 50px;
9 }
10
11 section .class-one .button {
12   color: #074e68;
13 }
```

Elaborate on what you want to discuss.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Semper quis lectus nulla at. Risus quis varius quam quisque id diam vel quam.

```
src/
├─ styles/
│   └─ base/
│       └─ _reset.scss
│       └─ _typography.scss
│   └─ components/
│       └─ _buttons.scss
│       └─ _cards.scss
│   └─ layout/
│       └─ _header.scss
│       └─ _sidebar.scss
│   └─ utilities/
│       └─ _helpers.scss
└─ main.scss
```


Variables simples

SCSS

```
$radius-base: 0.5rem;
```

Usage :

SCSS

```
.card {  
  border-radius: $radius-base;  
}
```

Maps (design tokens CSS)

SCSS

```
$z-index: (  
  header: 10,  
  sidebar: 20,  
  modal: 100  
);
```

SCSS

```
.header {  
  z-index: map-get($z-index, header);  
}
```

```
@mixin flex-center {  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}
```

SCSS

```
.modal {  
  @include flex-center;  
}
```

variables, maps et mixins.

les 3 principaux outils SASS

<https://sass-lang.com/>



Contact

nicolas.hersant@gmail.com

@NicoHersant // github || gitlab || linkedIn

onimaru6172 // discord

